



Cyber Security Educational Sandbox

Divyansh Mukati

25MEI10075

Guided by

Dr. A.V.R. Mayuri

VIT BHOPAL

2025

Deceleration

I hereby declare that the project titled “**Cyber-Security Educational Sandbox**” is an original piece of work completed by me under the guidance of **Dr. A.V.R. Mayuri**. This work has not been submitted to any other institution for academic credit, certification, or award.

Certificate

This is to certify that the project titled “**Cyber-Security Educational Sandbox**”, submitted by **Divyansh Mukati**, Reg. No. **25MEI10075**, is a bonfide record of work carried out during the academic year 2025 under my supervision.

Guide Signature: _____

HOD Signature: _____

Acknowledgement

I express my deep gratitude to my project guide, **Dr. A.V.R. Mayuri**, for providing continuous guidance, valuable suggestions, and motivation throughout the project. Their feedback helped me understand complex cyber-security concepts in a structured manner.

I would also like to thank the Computer Science & Engineering department for the resources and support provided. Additionally, I extend my thanks to my family and friends for their encouragement and help during the project development.

Abstract

Cyber-security is an increasingly important field as the digital world grows in complexity and vulnerability. However, beginners often struggle to study cyber-security practically due to the risks associated with experimenting on live systems.

This project, **Cyber-Security Educational Sandbox**, provides a safe, controlled, and isolated environment designed specifically for students to learn and practice essential security concepts. The sandbox includes three interactive modules:

1. **File Integrity Checker** – Detects unauthorized modifications in files using SHA-256 hashing.
2. **Network Monitor** – Displays system network activity, open ports, and connected devices.
3. **Simple Honeypot Simulation** – Logs connection attempts on a fake port to simulate attacker probing.

The project demonstrates real-world cyber-security principles using simple and accessible Python code, allowing beginners to gain hands-on experience without risking system damage.

Introduction

The modern digital landscape is dominated by data-driven systems, interconnected networks, and widespread automation. While these advancements have created immense opportunities, they have also exposed systems to a wide range of cyber threats including malware attacks, unauthorized access, phishing, port scanning, and system exploitation.

Understanding cyber-security tools is essential for students pursuing careers in the field. However, performing security experiments directly on a personal system may:

- Corrupt important data
- Affect system performance
- Cause network vulnerabilities
- Introduce malicious elements

To address this challenge, this project proposes a **Cyber-Security Educational Sandbox**, which acts as a safe playground for performing basic cyber-security experiments.

The sandbox includes modules that simulate:

- File tampering detection
- Monitoring active network connections
- Recording suspicious access attempts on ports

This project helps students visualize how cyber-security tools work internally while building a strong practical foundation.

Problem Statement

Cyber-security education requires practical exposure, but conducting activities such as port scanning, file monitoring, network sniffing, or handling suspicious connections can expose a system to significant risks.

The core problems are:

- Lack of a secure setup where students can practice freely
- High risk associated with running unknown scripts or monitoring tools
- Difficulty understanding internal processes of security tools without visualization
- Limited availability of beginner-friendly sandbox projects

Therefore, there is a strong need for a **simple, secure, and isolated sandbox** that lets students learn fundamental cyber-security concepts without harming their primary operating system.

Objectives

The project aims to:

1. Build a **safe and controlled sandbox** for cyber-security experiments.
2. Implement a **File Integrity Checker** to detect unauthorized changes.
3. Develop a **Network Monitor** to observe system-level network activity.
4. Design a **Honeypot** to log suspicious connection attempts and probing behavior.
5. Simplify cyber-security concepts for beginners through Python-based implementation.
6. Provide clear logs and outputs to help learners understand behavior step-by-step.
7. Create modular code that can be easily extended for future enhancements.

Literature Review

Various cyber-security tools and sandbox environments exist today:

Cuckoo Sandbox

- Used globally for analyzing malicious files.
- Highly advanced, but difficult for beginners.

FireEye Malware Analysis Tools

- Enterprise-level, expensive, and not accessible to students.

Virtual Box / VMWare

- Allow isolated environments, but setting up full VMs requires technical expertise.

Docker Containers

- Provide lightweight isolation but need command-line expertise and security understanding.

Most tools prioritize **professional analysis** rather than **beginner education**.

Therefore, this project focuses on:

- Simplified tools
- Clear outputs
- Easy-to-understand logic
- Zero risk to system
- Beginner-friendly design

This gap in the existing ecosystem is the motivation for building an educational sandbox.

Proposed System

The system is designed with three major modules:

1) File Integrity Checker

Purpose:

To detect if a file has been modified intentionally or unintentionally.

How it works:

- Takes a file as input
- Computes its **SHA-256 hash**
- Saves the hash
- On next run, compares the new hash with the stored value
- If mismatch → file tampering detected

Applications:

- Detecting malware infections
- Detecting unauthorized changes
- Ensuring file authenticity

2) Network Monitor

Purpose:

To observe network activity and open connections.

Features:

- Get local system IP
- List open ports
- Show active connections
- Log suspicious or unknown connections
- Helps students understand how data flows in/out of a device

Applications:

- Identifying unusual processes
- Learning about TCP/UDP
- Detecting suspicious network activity

3) Simple Honeypot Simulation

Purpose:

To simulate a fake vulnerable service.

Working:

- Creates a socket listening on a port
- If someone tries to connect → logs the attempt
- Helps understand port scanning behavior

Applications:

- Teaching intrusion detection
- Understanding attacker reconnaissance
- Demonstrating simple honeypot concepts

Methodology

7.1 File Integrity Checker

1. User selects the target file
2. Program computes SHA-256 hash
3. Hash stored in a text/database file
4. System re-checks the file later
5. New hash is compared
6. If changed → “tampering detected”
7. Log entry created

7.2 Network Monitor

1. Retrieves user's local IP address
2. Scans active network connections
3. Checks open ports using psutil or socket
4. Displays each connection:
 - Local address
 - Remote address
 - Port number
 - Connection status
 - Process ID
5. Logs unusual or unknown connections

7.3 Honeypot Simulation

1. Opens a TCP socket on a fake port
2. Waits for incoming connection

3. Logs the information:
 - IP Address
 - Port
 - Timestamp
4. Stores logs in honeypot_log.txt
5. Closes the socket safely

Implementation

Languages & Tools

- **Python 3.x**
- **hashlib** – File hashing
- **socket** – Network communication
- **psutil** – Reading system connections
- **datetime** – Logging timestamps

Platform

- Windows 10
- Linux (Ubuntu recommended)

Tested Output

- File Integrity Checker showing hash comparison
- Network Monitor listing connections
- Honeypot logging IP attempts

Testing

Test case	Action	Expected output	Result
File change	Modify the file manually	Detect change	Passed
No change	Keep file unchanged	No alert	Passed
Network monitor	Scan ports	Show active ports	Passed
Honeypot access	Connect via telnet	Log IP	Passed
Invalid input	Provide wrong file path	Show error	Passed

Results

The sandbox successfully:

- Detected file tampering using secure hashing
- Displayed real-time network connections
- Logged probing attempts on the honeypot port
- Helped visualize internal behavior of security tools
- Worked across multiple operating systems
- Provided a safe environment with no risk to system

Conclusion

The **Cyber-Security Educational Sandbox** fulfills the purpose of creating a controlled environment for students to learn basic security mechanisms. It bridges the gap between theoretical learning and practical cyber-security exposure.

The project's modular approach makes it easy to extend and adapt for more advanced topics, making it a valuable educational tool.

Future Enhancements

1. Real-time dashboard (Flask-based web UI)
2. Malware behavior simulator (safe version)
3. Packet sniffer using Scapy
4. System log analyzer
5. Email/SMS alerts for suspicious activity
6. GUI interface instead of CLI

References

- Python Official Documentation
- psutil Library Documentation
- Cyber-Security Textbooks
- OWASP Security Guides
- Online network security articles
- File hashing algorithm specifications
- Chat generative pre-trained transformer